

0.80. x et y rangés ensemble : struct

`struct Position { uint8_t x, y; }` : la colonne et la ligne d'une lettre sous un seul nom, `joueur.x` et `joueur.y`.

1. Ce qu'il faut comprendre

Deux variables qui vont toujours ensemble — la colonne et la ligne d'une lettre — peuvent être rangées **sous un seul nom**.

Ce qui est nouveau ici : `struct`. `struct Position { uint8_t x, y; };` crée un **nouveau type**, `Position`, qui contient deux octets : `x` et `y`. Puis `Position joueur;` crée une variable de ce type.

Le point `.` atteint une partie : `joueur.x` est la colonne du joueur, `joueur.y` sa ligne. On s'en sert comme de n'importe quelle variable : `joueur.x = 10;`, `deplace_croix(joueur.x, joueur.y, ...)`.

Pourquoi c'est utile : avec plusieurs lettres, `Position joueur;` et `Position ennemi;` se lisent mieux que `x`, `y`, `xb`, `yb`. On ne mélange plus les variables d'une lettre avec celles d'une autre.

La console range toujours la position dans `joueur.x` et `joueur.y` : ce qu'elle sait faire avec une variable (le 0.23), elle le sait avec une partie de `struct`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1241 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	65 o	← la console, à l'allumage — la mise en route de la console
deplace_croix()	217 o	← main() — une fonction fournie par le langage
main()	67 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,

DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :

nombre()	× 2	22 o sur place, dans main()	+ EcrireNombre (33 o)
bouton()	× 4	20 o sur place, dans deplace_croix()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans deplace_croix()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans deplace_croix()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()	

Tes fonctions appelées :

deplace_croix() × 1 21 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le nouveau type, morceau par morceau :  
//  
// struct Position { uint8_t x, y; };  
// |           |           |  
// |           |           +-- ses parties : deux octets, x et y  
// |           +----- son nom : Position  
// +----- « struct » : on crée un type à nous  
  
#include <deplace_croix> // une tuile qui suit la croix, case par case  
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A  
#include <nombre>        // écrit un nombre en chiffres
```

2. UN ENREGISTREMENT — POSITION

```
struct Position { uint8_t x, y; };
```

3. UNE VARIABLE — JOUEUR

```
Position joueur; // une variable de type Position : joueur.x et joueur.y
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

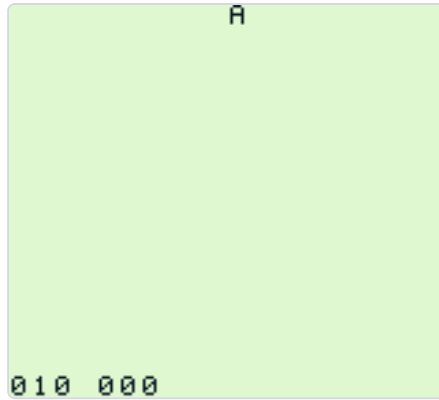
```
int main() {  
    joueur.x = 10; // le point atteint une partie : la colonne...  
    joueur.y = 0; // ...et la ligne. Départ : (10, 0).  
  
    while (true) {  
        image();  
        deplace_croix(joueur.x, joueur.y, ALPHABET[0], 250); // la console range dans joueur.x, joueur.y  
        nombre(0, 17, joueur.x);  
        nombre(4, 17, joueur.y);  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A en haut de l'écran, qui suit la croix ; sa position, `joueur.x` et `joueur.y`, en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
- ✓ la croix le déplace : `joueur.x` diminue, `joueur.y` augmente (6, 2)
- ✓ il est à la place que disent `joueur.x` et `joueur.y`

Ensuite : **0.80.1. x et y rangés ensemble : struct — la boucle dans un fichier** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++